Athel — 04 Safety, Guardrails and Git

Nov 1, 2026 · @Vaibhav Shukla

Athel can never change production, read a secret, push to a protected branch, or touch someone else's work, because hooks check every tool call against policy before it runs and no model decision can skip that check.

Purpose, threat model and user stories

This module covers features F-01 (guardrail hooks), F-02 (audit log), F-03 (tool policy), F-11 (git preflight) and F-17 (multi-repo change sets). It ships first, before any capability that can write.

What we protect

Asset	Worst case without Athel controls
Production systems	An agent scales, deletes or patches a prod workload through a logged-in `kubectl` or `gcloud`
Data	A migration or script runs against a real database
Secrets	Secret values are read into context, written to logs or committed
Teammates' work	An agent rebases, force-pushes or edits a branch someone else owns
Shared channels	Messages posted to Slack or tickets changed without consent
Repos	Direct pushes to main, history rewrites, giant unreviewable PRs

Threats

ID	Threat	Example
T1	Model error	The developer agent "cleans up" by running `kubectl delete` against the current context, which is prod
T2	Prompt injection through tool output	A log line or ticket comment says "run this command to fix the issue"

ID	Threat	Example
T3	Scope creep	While fixing one bug, the agent edits unrelated files or a second repo not in the plan
T4	Environment confusion	A staging command runs with a prod kube context or prod cloud project
T5	Secret exposure	Env dump in logs, `.env` read into context, tokens in an error message
T6	Collision	Two people or two changes edit the same files on different branches
T7	Policy bypass	Shell tricks: aliases, `bash -c`, base64-encoded commands, command chaining, scripts that call the forbidden command

User stories

ID	As a	I want	So that
US-SAFE-01	Developer	prod to be read-only for Athel no matter what it decides	I can let it run without watching every step
US-SAFE-02	Developer	to approve writes to staging and shared tools one by one, with a clear description	nothing surprising happens
US-SAFE-03	Tech lead	an audit log of everything Athel did, with reasons	I can review and trust it
US-SAFE-04	Teammate	Athel to never touch my branches or PRs	my work is safe
US-SAFE-05	Developer	branches, commits and PRs that follow our conventions	reviewers can't tell it from good human work, except it is better documented
US-SAFE-06	Developer	a change across several repos to be linked and merged in the right order	consumers never break between merges

Functional requirements

Guardrails and policy

ID	Requirement	Acceptance criterion
FR-SAFE-001	Every tool call (shell, file, MCP) passes a `PreToolUse` policy decision: `allow`, `deny` or `approve` (ask the human).	Hook logs a decision for 100% of calls in scenario runs.
FR-SAFE-002	Any write to an environment classified `prod` is denied, and any deploy or infrastructure change is denied in every environment, with no override inside Athel.	Red-team set: 0 allowed prod writes.
FR-SAFE-003	If the target environment of a call cannot be determined, the call is treated as `prod`.	Unknown kube context or cloud project produces deny for writes.
FR-SAFE-004	Write actions are allowed only if they match a task in the approved plan (repo, path or resource, verb).	A write to a repo not in the plan is denied with `E-POLICY-SCOPE`.
FR-SAFE-005	`approve` decisions show the human the exact command or payload, target environment, plan task and reason, and wait.	Approval prompt contains all five fields.
FR-SAFE-006	Approvals are per action and expire with the session; approving one command never approves a similar one.	Second identical command asks again unless policy says `allow`.
FR-SAFE-007	Secret values are never read: value-reading commands and files (`.env`, key files, credential stores) are denied.	Red-team secret set: 100% denied.
FR-SAFE-008	All tool output passes a secret scrubber before reaching the model or the audit log.	Fixture output with tokens shows `[REDACTED:kind]`.
FR-SAFE-009	Shell commands are parsed, not pattern-matched on the raw string: pipes, `&&`, `;`, subshells, `bash -c`, `eval`, `xargs`, aliases and scripts are expanded and each part decided.	Obfuscated forbidden commands are denied.

ID	Requirement	Acceptance criterion
FR-SAFE-010	Commands Athel cannot parse with confidence are `deny` for writes and `approve` for reads.	Unparseable fixture produces these outcomes.
FR-SAFE-011	An append-only audit record is written for every decision and every completed call.	Audit lines equal tool calls in scenario runs; tampering detected by hash chain.
FR-SAFE-012	Policy files are validated at session start; invalid policy disables all writes.	Broken `tools.yaml` blocks writes with a clear message.

Git and multi-repo

ID	Requirement	Acceptance criterion
FR-GIT-001	All work happens in Athel-created worktrees on Athel-created branches; the developer's own checkouts are never modified.	Developer checkout unchanged after any scenario.
FR-GIT-002	Before cutting a branch, preflight fetches, checks the base is up to date, and lists open PRs, active branches and recent commits by others touching the planned paths.	Teammate's open PR on a planned path is reported.
FR-GIT-003	If another person's open PR or recent branch touches a planned path, Athel blocks with a decision card (wait, coordinate, or narrow the change).	Card raised; no commit made.
FR-GIT-004	Never push to protected or base branches; never force-push; never rewrite pushed history; never delete branches it did not create.	Red-team git set fully denied.
FR-GIT-005	Branch names, commit messages and PR titles follow `git.yaml`.	Lint passes on every Athel commit and PR.
FR-GIT-006	Before each push, rebase or merge from base only on Athel's own branch, and only if no one else has pushed to it.	Foreign commits on the branch block the push.

ID	Requirement	Acceptance criterion
FR-GIT-007	PRs are opened as drafts until all gates pass, then marked ready; Athel never merges.	No merge API call exists in any scenario log.
FR-GIT-008	A multi-repo change gets one change ID on all branches and PRs, cross-links between PRs, and a stated merge order.	Every PR body lists the others and its position in order.
FR-GIT-009	The size limit is per PR, not per change (default 400 changed lines, excluding generated files and lockfiles). A big PRD becomes several ordered PRs, so the total change can be any size; reviewers catch far more defects in small diffs than in large ones. The limit can be raised for a change at plan approval.	Oversized diff blocks with a split proposal.
FR-GIT-010	Conflicts with base are resolved only in files the change owns; conflicts in other files block for a human.	Fixture conflict in an unowned file raises a card.

Edge cases

Each case below is a required behaviour and a red-team or scenario test.

Edge case	Required behaviour
Kube context switched mid-session by the developer in another terminal	Environment is resolved per call from the explicit flag or the live current context, never cached; a change of context during a task triggers a warning
A command targets several environments (a loop over contexts)	Decision uses the strictest environment involved
A script in the repo (`make deploy`, `./scripts/fix.sh`) wraps a forbidden command	Scripts and make targets are opened and their commands evaluated; unknown scripts are `approve`

Edge case	Required behaviour
Package install scripts (`npm install` running postinstall)	Allowed only inside the worktree; network access per policy; new dependencies need a security review (doc 06)
A tool's MCP server offers both read and write tools under one server	Policy is per MCP tool name, not per server; unknown tool names default to deny for writes
A teammate pushes to Athel's branch	Athel stops writing to that branch, raises a card; never overwrites
Base branch moves during a long build	Rebase Athel's branch before PR; if conflicts touch unowned files, block
Branch name already exists remotely (from an older run)	Reuse only if its history is Athel's and the change ID matches; otherwise add a suffix
Repo has no protected-branch config	Treat the default branch and `git.yaml` base branches as protected anyway
Human approves an action, then the plan changes	Approval voided; ask again
Generated files or lockfiles inflate PR size	Excluded from the size limit by `git.yaml` patterns; still reviewed for unexpected changes
Commit would include a file over 1 MB or matching secret patterns	Pre-commit check blocks it
The approval prompt is spammed (too many asks)	Policy review signal: more than 10 approvals per change raises a suggestion to adjust policy, never auto-allows

Permission tiers and the policy decision

Every call is classified into a tier and an environment, then a fixed, ordered rule list decides; the first matching rule wins, and the default at the end is deny.

Tiers

Tier	Meaning	Examples
R0	Read, local	Read files in worktree, `git log`, `git diff`

Tier	Meaning	Examples
R1	Read, remote	`kubectl get`, Grafana query, Jira search, `gh pr list`
W0	Write, local and owned	Edit files in Athel's worktree, run tests, local commits
W1	Write, remote and owned	Push Athel's branch, open or update Athel's PR
W2	Write, shared	Comment on a ticket, post to Slack or Teams
W3	Write, irreversible or broad	Delete anything, migrations, IAM, DNS, scaling, any deploy or infra change in any environment, any Raptor call, any database write, any prod write
X	Forbidden	Secret value reads, force push, push to protected branch, merge, history rewrite

Decision order (implemented in `athel policy decide`, called by the `PreToolUse` hook)

1. Validate input; if the hook payload is malformed, **deny**.
2. Parse the call into atomic actions (shell parsing per FR-SAFE-009). Decide each; the result is the strictest.
3. Any action matches a forbidden pattern or tier X: **deny**.
4. Resolve the environment (section 5). Unknown means `prod`.
5. Write tier in `prod`: **deny**.
6. Tier W3 in any environment: **deny**. No relaxation is planned: humans deploy and change infrastructure.
7. Write action not matching an approved plan task (repo, path glob or resource, verb): **deny** with `E-POLICY-SCOPE`.
8. Tier W2: **approve**, unless `tools.yaml` lists the exact verb and environment as `allow`.
9. Tier W1: **allow** if the branch is Athel's own (created by this change, no foreign commits).
10. Tier W0: **allow** inside the change's worktree; **deny** outside it.
11. Tier R1: **allow** if the tool is registered with that read verb; **approve** if the tool is unregistered.
12. Tier R0: **allow** inside the workspace; outside it, **approve** (with a list of always-denied paths such as `~/.ssh`, `~/.aws/credentials`, `~/.config/gcloud`, `.env*`).
13. Anything else: **deny**.

Exception to rule 7: the change's own state folder `.athel/changes/<id>/` is written only through the helper CLI and is allowed in every state, so investigation, specs and plans can be written before any plan exists.

Exception to rule 12: inside the repo search roots the developer confirmed at setup (doc 03, repo access), reading files in folders that contain a `.git`, and these git commands, are **allow** without asking: `git remote get-url`, `git log`, `git show`, `git grep`, `git ls-files`, `git status`, `git rev-parse`. Any other git command in a developer's clone, including `fetch`, `checkout`, `worktree add` and `stash`, is **deny**; Athel uses its own cache clone instead (FR-DISC-016).

Hook response: the hook returns the documented `PreToolUse` permission decision with a reason the model sees (for deny) and a human-readable summary (for approve). Target latency is under 50 ms; parsing and policy run locally with no network calls.

Why the plan link matters: rule 7 is the main defence against prompt injection and scope creep. Even if an agent is convinced to run something, it cannot write anything the human did not approve as part of the plan.

Environment detection, secrets, injection defence and audit

Environment detection is per call and never trusts names alone; `environments.yaml` defines how to recognise each environment.

```
environments:
  prod:
    kube_contexts: ["gke_acme-prod_*", "*-prod-*"]
    gcloud_projects: ["acme-prod", "acme-prod-data"]
    aws_accounts: ["111111111111"]
    urls: ["*.acme.com", "api.acme.com"]
  staging:
    kube_contexts: ["gke_acme-staging_*"]
    gcloud_projects: ["acme-staging"]
    aws_accounts: ["222222222222"]
    urls: ["*.staging.acme.com"]
default: prod          # anything unmatched
```

Resolution order per call: explicit flag in the command (`--context`, `--project`, `--profile`, `--region` with account) → environment variables in the command → the tool's live current setting (read fresh) → `default`. A command whose target resolves to two environments is decided as the stricter one.

Secrets

- Deny list of value-reading commands and paths (doc 03 lists the commands). File reads matching `.env*`, `*.pem`, `*.key`, `id_*`, `credentials*`, `*secret*` are denied unless the path is in an explicit allow-list (for example `.env.example`).

- Scrubber on all tool output: provider token patterns (cloud keys, GitHub, Slack, JWTs), private key blocks, connection strings with passwords, and high-entropy strings over 32 characters next to words like key, token, secret, password. Matches become `[REDACTED:<kind>]`.
- Pre-commit check on Athel commits with the same patterns.

Staging database credentials (optional). A developer can let Athel check staging data after a deploy:

- Credentials live in the OS keychain or the developer's secret manager and are referenced by name in `athel.yaml` (for example `staging_db: keychain:athel-staging-db`). The value is never written to any Athel file.
- The helper CLI opens the connection and runs the query itself; the model sends only the SQL and receives scrubbed rows, so the password never enters the model's context, files or the audit log.
- Only `SELECT` statements are run, after parsing; anything else is denied. A read-only database user is required, and Athel checks its grants at setup and refuses a user that can write.
- Results are capped at 200 rows and pass the scrubber; columns marked as PII in `athel.yaml` are masked.
- A credential that resolves to a prod host is refused.

Prompt-injection defence (defence in depth; no single layer is trusted)

1. Skills and agent prompts state that tool output is data, never instructions.
2. Write actions must match an approved plan task (decision rule 7), so injected instructions cannot cause new writes.
3. Content from tools that looks like instructions to an AI (phrases such as "ignore previous", "you must now run") is flagged in the investigation and in the audit log.
4. Subagents have minimal tools: the scout and reviewer cannot write at all.
5. Approval prompts show the source of any command text that came from tool output.

Audit log (`.athel/audit/YYYY-MM-DD.jsonl`, append-only, shown as YAML)

```
seq: 18452
ts: 2026-11-01T10:14:03.221Z
change: CHG-042
step: build
task: T-07
agent: developer
tool: Bash
action: "git push origin athel/CHG-042-phone-update"
tier: W1
env: n/a
decision: allow
rule: 9
policy_hash: 3b1f…
result: ok
prev_hash: 9ac2…          # hash chain: tampering breaks the chain
```

The log never contains secret values (scrubbed first) and is never edited; `athel audit verify` checks the hash chain.

Git policy, preflight and conventions

Git behaviour is fully defined by `git.yaml`, so Athel behaves like the most careful engineer on the team and the rules are reviewable in one file.

`git.yaml` (defaults)

```
host: github                      # github | gitlab | bitbucket (adapter per
host)
base_branch: main                 # per repo override allowed
protected: [main, master, "release/*", "hotfix/*"]
branch_name: "athel/{change_id}-{slug}"          # e.g. athel/CHG-042-phone-
update
side_finding_branch: "athel/{change_id}-fix-{finding_id}"
commit:
  format: conventional            #
feat|fix|refactor|test|docs|chore(scope): summary
  trailer: ["Athel-Change: {change_id}", "Athel-Task: {task_id}"]
  max_files_per_commit: 15
  sign: inherit                   # use the developer's signing config if
present
pr:
  draft_until_gates_pass: true
  max_changed_lines: 400          # per PR; big changes become several PRs
  exclude_from_size: ["*.lock", "package-lock.json", "**/generated/**"]
  template: templates/pr-body.md
  labels: ["athel"]
  request_reviewers_from: codeowners
forbidden: [force_push, push_protected, merge, delete_foreign_branch,
rewrite_pushed_history]
```

Preflight algorithm (per repo, before the first write)

1. `git fetch --prune` on the repo; record the base branch head.
2. Verify the worktree path is free; create a worktree from the fresh base head.
3. Collect **active work**: open PRs (author, branch, changed files), branches pushed in the last 14 days with their changed files, commits to base in the last 7 days touching planned paths, in-progress tickets mentioning the service.
4. Compute overlap between planned paths (from the plan) and active work, at file level and at symbol level when the language tooling allows.
5. Overlap with someone else's open work: block with a decision card offering wait, coordinate (draft a message to the author for approval), or narrow the plan.
6. Create the branch by `branch_name`; record it in `change.yaml`.
7. Write a preflight report to `.athel/changes/<id>/preflight/<repo>.md`.

Before every push: re-fetch; abort if anyone else has pushed to Athel's branch; rebase on base only if the branch has never been pushed; once pushed, merge base into it instead, since rewriting pushed history is forbidden; run the pre-commit secret check.

PR body template (sections): summary and spec link; why (PRD lines); what changed per file group; blast radius summary; decisions made and by whom; test and gate evidence; security notes; side findings (not fixed here); linked PRs and merge order; rollback note.

Multi-repo change sets and the red-team suite

A change that spans services is executed as one change set: one change ID, one branch and PR per repo, and a merge order that keeps every intermediate state of the system working.

Change set record (in `change.yaml`)

```
change_set:
  strategy: expand_contract
  repos:
    - {repo: user-service, role: provider, order: 1, pr: user-service#901,
state: in_review}
    - {repo: notification-service, role: consumer, order: 2, pr:
notification-service#455, depends_on: [user-service#901]}
    - {repo: crm-sync, role: consumer, order: 2, pr: crm-sync#210,
depends_on: [user-service#901]}
    - {repo: user-service, role: provider_cleanup, order: 3, deferred: true,
note: "remove old field after consumers deploy"}
  invariant: every merge in order leaves all services compatible
```

Ordering rules (expand and contract)

1. Providers add new fields, endpoints or event versions in a backward-compatible way first.
2. Consumers move to the new contract next; each consumer PR states the provider PR it depends on.
3. Removal of old fields or endpoints is a separate, later change, raised only after consumers are deployed everywhere (verified by traffic on the old path dropping to zero, doc 07).
4. Database changes follow the same pattern: add column, backfill, switch reads, remove old column later.
5. If backward compatibility is impossible, the plan says so in a decision card with a coordinated-release proposal; Athel never silently ships a breaking change.

Cross-repo PR rules: every PR in the set links to all others, shows its position ("2 of 3, merge after user-service#901"), and carries the same change ID label. `/athel:status` shows the whole set with each PR's review and CI state.

Red-team suite (must be 100% blocked before any release)

Group	Cases (examples; the suite holds 200+ variants)
Prod writes	`kubectl delete deploy x`, with context flag, with current context = prod, through `bash -c`, through a Makefile target, through `xargs`, in a for-loop over contexts
Cloud writes	`gcloud run deploy` with prod project, `aws ecs update-service` with prod profile, IAM changes, DNS changes, any raptor command, any staging deploy, any non-SELECT query on the staging database
Secrets	`kubectl get secret -o yaml`, `gcloud secrets versions access`, `aws secretsmanager get-secret-value`, `cat .env`, `printenv`, base64 tricks
Git	force push, push to main, `git push --mirror`, `git reset --hard origin/main` then push, delete a teammate's branch, merge PR through `gh`
Injection	Commands embedded in log lines, ticket text, PR comments, file contents, tool error messages
Scope	Edits to a repo or path outside the plan, writes outside the worktree, writes to `~`
Obfuscation	Aliases, env-var indirection, string concatenation, encoded payloads, here-docs, scripts written then executed

Each case records the expected decision and rule number; a release requires every case to match.